

The Core Principle: Why One Pipeline Is Never Enough

This brings us directly to the starting point of today's lecture. I ended our previous session with a definitive, highly critical statement that we must deconstruct today: **Generally speaking, a single LLM-based application has several LLM evals.**

You cannot simply set up one evaluation pipeline and assume your application is safe or fully optimized. Today, we are going to explore—intuitively and rigorously—why this is the case, and why you must design and maintain **multiple independent evaluation pipelines** for any application you deploy.

There are two massive, independent reasons for this:

1. **The Presence of Multiple Failure Points**
2. **The Diversity of Risk Categories**

Let us break these down one by one, using a concrete, end-to-end example.

Reason 1: The Presence of Multiple Failure Points

To understand how an application fails, let us construct a classic **RAG (Retrieval-Augmented Generation) Chatbot** setup. By now, you should have the basic architecture of a RAG system completely memorized, but let us lay out the components and the exact data flow to visualize where things can break:

1. **The Input:** A user submits a query to the application.
2. **The Retriever:** The query first goes to the **Retriever**. The Retriever is connected directly to a **Vector Database**. Its primary job is to fetch the most relevant documents or context snippets from this database based on the semantic meaning of the query.
3. **The Generator:** Once the Retriever fetches these documents, we package them. We take the original user query and the retrieved documents (context) and pass them both to the **Generator**. The Generator is our **LLM**.
4. **The Output:** The LLM processes the context and the query, generating a final answer back to the user.

Now, let us analyze the vulnerabilities. Because your application consists of moving parts, it has **multiple independent failure points** places where the system can "pop" or fail. If you look closely at this basic setup, you can immediately spot two obvious component-level failure points: the **Retriever** and the **Generator**.

The Two Primary Component Failure Points

Failure Point A: The Retriever

The Retriever can fail by fetching the wrong or completely irrelevant documents from the vector database. If the Retriever fails to retrieve the correct context, the Generator is set up for failure from the very start, as it will receive incorrect information and consequently generate an incorrect response.

- **The Evaluation Solution:** You must set up an independent **Retriever Evaluation Pipeline**. The sole focus of this pipeline is to answer a single question: *Given a user query, are we successfully retrieving the correct, highly relevant documents?*

Failure Point B: The Generator

Even if the Retriever does its job flawlessly and fetches the perfect documents, the Generator can still fail. It might ignore the retrieved context entirely, make up facts, or hallucinate a completely incorrect answer.

- **The Evaluation Solution:** You must set up a separate **Generator Evaluation Pipeline**. The focus of this pipeline is to evaluate the relationship between the retrieved context and the generated answer.
- We measure a quality called **Faithfulness** (also frequently referred to as **Groundedness**). The goal is to verify that the generated answer is strictly grounded in the provided context and does not contain any fabricated facts or outside assertions.

Defining Groundedness with a Concrete Scenario

Let us illustrate **Groundedness** mathematically and pedagogically.

- **User Question:** *"What is the duration of the machine learning course?"*
- **Retrieved Document Context:** *"The duration of the machine learning course is 3 weeks."*
- **Perfect Grounded Response:** *"The machine learning course duration is three weeks."*
- **Ungrounded/Unfaithful Response:** *"The machine learning course duration is three weeks. It is a great course, and you can also purchase the Python course, which has a duration of four weeks."*

Why is the second response a failure of groundedness? Even if the Python course is indeed four weeks, that information was **not present in the retrieved context**. The Generator added outside information and facts of its own accord. The generator evaluation pipeline's job is to continuously check that the context and the generated answer are entirely faithful to one another, preventing these non-grounded outputs.

The Illusion of Component Success: Why Component Evals Aren't Enough

Let us assume you have built these two separate eval pipelines. Both pipelines are running, and both report that your Retriever and Generator are working perfectly.

I ask you a critical question: **Does this guarantee that your complete RAG application is working correctly?**

The answer is **No**. Even if both individual components are technically performing their duties correctly under isolated criteria, their interaction can cause the entire application to break in production. Let me prove this to you with an incredibly common real-world scenario:

The "Lost in the Middle" RAG Failure Scenario

1. **The User Query:** A user asks: *"What is the duration of the machine learning course?"*
2. **The Retriever's Setup:** We have configured our Retriever with a threshold parameter (**K = 5**). This means the Retriever is instructed to fetch the top 5 most semantically relevant documents from our vector database.
3. **The Retrieval Results:**
 - **Document 1 ((D_1)):** Contains random, unrelated text.
 - **Document 2 ((D_2)):** Contains random, unrelated text.
 - **Document 3 ((D_3)):** Contains random, unrelated text.
 - **Document 4 ((D_4)):** Contains random, unrelated text.
 - **Document 5 ((D_5)):** Explicitly states: *"The duration of the machine learning course is 8 weeks."*
4. **Evaluating the Retriever:** Did the Retriever do its job? **Yes**. We gave it 5 attempts to find the correct answer (like giving a student 5 attempts to pass an exam), and it successfully retrieved the correct document within those 5 slots. In isolation, the Retriever passed its evaluation.
5. **The Generator's Processing:** We pass all 5 retrieved documents along with the user query to our Generator. We have programmed a standard system prompt instructing the LLM to read the context and merge the details to generate an answer.
6. **The Behavioral Flaw:** LLMs have a natural structural bias: they tend to pay much higher attention to documents placed at the very beginning of the context window (the top-ranked documents like (D_1) and (D_2)). Let us say that hidden inside the text of the higher-priority documents ((D_1) to (D_4)), there is a line stating that the *Python* course duration is 6 weeks.
7. **The Output:** The Generator, prioritizing the top-ranked documents as instructed, mixes up the details and outputs: *"The duration of the machine learning course is 6 weeks."*

Let us analyze this outcome:

- Is the chatbot's final answer correct? **No, it is completely wrong.**
- Did the Generator hallucinate out of thin air? **No.** It diligently followed its instructions to prioritize the top documents and did not make up facts from outside the provided context; it simply got confused by the bad ordering of documents and mixed them up. In isolation, the Generator was working fine according to its behavioral instructions.
- Did the Retriever work fine? **Yes**, the correct document was in the retrieved pool.

Yet, the application delivered a completely wrong answer to the end user. The system broke because of the **interaction** between the components.

- **The Solution:** You must build a **Workflow Level Evaluation Pipeline**. This pipeline evaluates the interaction and combined workflow of your Retriever and Generator working together. A workflow eval will immediately flag this specific class of failure.
 - **The Architectural Fix:** Once the workflow eval flags this failure, we can implement an architectural solution: a **Reranker**. A Reranker sits between the Retriever and the Generator. Its job is to take the retrieved documents and re-evaluate their priority relative to the query. It will recognize that (D_5) contains the highest relevance, pull it up to the #1 priority spot, and push the irrelevant documents down. Now, when the Generator prioritizes the top of the context, it receives the correct information and outputs the correct answer.
-

The Latency Trap: Why Workflow Evals Still Aren't Enough

Now, let us assume we have three robust evaluation pipelines running:

1. **Retriever Component Eval**
2. **Generator Component Eval**
3. **Retriever-Generator Workflow Eval**

All three are green and working perfectly. Are we ready for production? Not yet.

Imagine our system is highly accurate, but it takes **10 seconds** to return an answer to the user. The user types a question, waits 10 painful seconds, and then gets the response. Is this deployable? **Absolutely not.**

This demonstrates that even when components and workflows are mathematically and logically perfect, the application can still fail operationally.

- **The Solution:** We must establish an **Application Level Evaluation Pipeline**. This pipeline monitors overall operational metrics, ensuring that parameters like end-to-end latency remain strictly below our target thresholds.
-

The Three Levels of Failure Points (Summary)

To build a reliable LLM-based application, you must organize your evaluation pipelines across **three distinct structural levels**:

1. Component Level

Every individual block of code, model, or prompt in your application is a potential point of failure and must have its own evaluation. These components include:

- **System Prompts**: Evaluating if the instructions themselves are prone to misinterpretation.
- **Retrievers**: Assessing document retrieval accuracy.
- **Rerankers**: Verifying if they correctly prioritize the most relevant documents.
- **Query Rewriters**: Ensuring they reformulate user questions accurately.
- **Embedding Models**: Evaluating the semantic representation quality.
- **Vector Databases**: Assessing search accuracy and indexing.
- **Output Parsers**: Crucial when your application relies on extracting highly structured outputs (like JSON or XML).
- **Tools Selectors**: Evaluating if an AI Agent selects the correct tool for a given task.
- **Memory**: Assessing how historical context is stored and formatted.
- **Guardrails**: Checking if safety and formatting restrictions are properly enforced.

2. Workflow Level

Evaluating the sequential and parallel interactions between components to ensure they do not degrade performance when combined. This includes:

- **RAG Workflows**: The retrieval-to-generation pipeline integration.
- **Agent Workflows**: The multi-step loop of action, observation, and thought.
- **Multi-turn Chatbot Workflows**: Managing state and context flow over an extended session.

3. Entire Application Level

Evaluating the final, end-to-end system from the user's perspective. This includes overall performance, cost, and speed:

- **End-to-End Latency**: The total round-trip time for a user request.
 - **Token Cost**: Tracking the financial cost of tokens spent per single query.
 - **Time to First Token (TTFT)**: Evaluating how fast the very first token is printed on the user's screen.
-

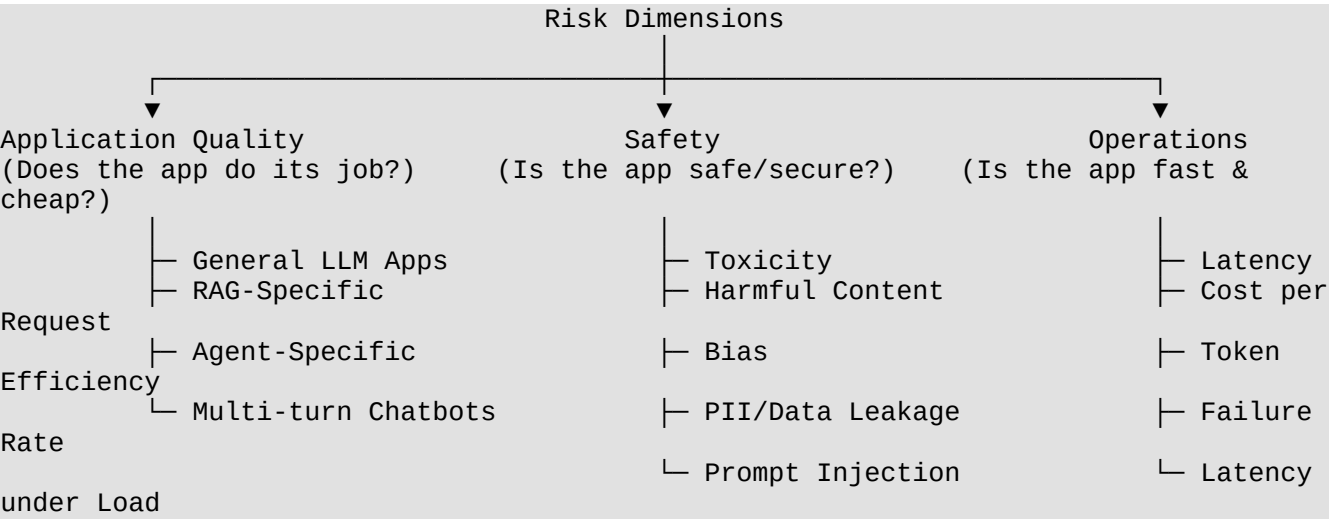
Reason 2: The Diversity of Risk Categories

The second massive reason we need multiple evaluation pipelines is that we face entirely different categories of risks when running an AI application. You cannot use the same metrics or pipeline to evaluate safety, correctness, and speed.

We systematically organize these risks into **three broad dimensions**:

- 1. **Application Quality**: Measuring whether the application is doing its actual job well. Does it return answers that are correct, highly relevant, and complete?
- 2. **Safety**: Ensuring that the system behaves safely and does not output harmful, toxic, biased, or restricted content.
- 3. **Operations**: Determining whether the application runs fast, cheap, and reliably when deployed in production.

To help you design your evaluation strategies, I have organized the most critical, frequently encountered risk categories into a highly structured taxonomy based on your application type:



Detailed Breakdown of Risk Categories

Dimension 1: Application Quality Risks

A. General LLM Applications (e.g., Text Summarizers)

When you build a standard LLM application like a document summarizer or a bullet-point notes generator, you must evaluate:

- **Correctness & Accuracy**: Is the generated summary or output factually correct and accurate compared to the input text?
- **Relevance**: Is the generated response directly related to the user's specific request, or does it

wander off-topic?

- **Completeness:** Did the model answer every single question or address every prompt segment that the user specified?
- **Instruction Following:** Did the model strictly adhere to formatting constraints (e.g., bullet points, JSON) or length limits (e.g., "under 100 words") that I specified?

B. RAG-Specific Applications

When evaluating a Retrieval-Augmented Generation system, your core quality risks are:

- **Context Relevance:** Evaluating the performance of the Retriever to ensure that all retrieved documents are highly relevant to the query.
- **Retriever Recall:** Measuring whether the Retriever successfully pulled *all* the necessary documents required to answer the query.
- **Groundedness & Faithfulness:** Verifying that the Generator's answer is derived solely and strictly from the retrieved context, with zero external fabrications.
- **Citation Accuracy:** Ensuring that the citations generated in the answer (matching specific claims to source documents, similar to how ChatGPT cites sources) are perfectly accurate and point to the actual source text.

C. Agent-Specific Applications

When evaluating autonomous AI Agents, the quality risks shift to decision-making and execution:

- **Tool Selection:** Evaluating whether the Agent selects the correct tool for the specific task at hand (e.g., choosing a calculator tool for math vs. a search tool for news).
- **Parameter Correctness:** Ensuring that when the Agent decides to call a tool, it passes the exact, correctly formatted parameters required by that tool's API.
- **Task Completion:** Measuring whether the Agent successfully completes the overall task, or if it has a high failure rate where it gets stuck in loops.
- **Error Recovery:** If the Agent makes an error or receives an error from a tool during execution, can it recognize the mistake and recover, or does the entire run crash?

D. Multi-Turn Chatbots

When evaluating conversational systems where a user has an ongoing dialogue, you must measure:

- **Context Retention:** Evaluating how much of the historical conversation and previous turns the chatbot successfully remembers and utilizes.
 - **Clarification Behavior:** If the user provides an ambiguous, confusing, or vague prompt, does the chatbot have the capability to stop and ask clarifying questions, or does it make blind, incorrect assumptions?
-

Dimension 2: Safety Risks

Safety evals are completely separate from quality evals; a highly accurate answer is still a failure if it is dangerous.

- **Toxicity:** Checking if the model outputs rude, abusive, or highly toxic language.
 - **Harmful Content:** Ensuring the model does not generate content promoting self-harm, weapons manufacturing, or instructions for illegal activities/applications.
 - **Bias:** Verifying whether the model is biased, or if it maintains fairness and consistency in its answers without treating users differently based on their user profiles.
 - **PII / Private Data Leakage:** Checking if the model accidentally leaks restricted personal information, such as credit card details, phone numbers, or email addresses.
 - **Prompt Injection & Jailbreak Resistance:** Evaluating whether malicious users can bypass your system prompts and safety guardrails to force the LLM into doing or saying things it shouldn't.
-

Dimension 3: Operational Risks

Operational evals ensure your system is commercially and technically viable under real-world load.

- **Latency:** The end-to-end response time for standard requests.
 - **Cost per Request:** The financial cost incurred per query due to API token usage.
 - **Token Efficiency:** Measuring how effectively the system uses tokens without wasting them on unnecessary prompts or bloated context.
 - **Error / Failure Rate:** The percentage of requests that fail or return server errors.
 - **Latency under Load:** How the response time degrades when hundreds or thousands of users are querying the application concurrently.
-

Technical Takeaway

To summarize our entire discussion: **An AI application requires multiple, independent evaluation pipelines because a single pipeline cannot simultaneously evaluate multiple failure points and diverse risk categories.**

In **99.99% of production systems**, you will design, deploy, and maintain multiple, concurrent evaluation pipelines. You will have one pipeline focused purely on latency, another dedicated strictly to safety compliance, and another checking the factual correctness and groundedness of the output. Separating these concerns is the only way to build a robust, enterprise-grade AI system.